

Pretraining Objective and Language Modeling

pig7selene

September 5, 2026

Abstract

本章从 decoder-only Transformer 的输出 logits 出发，形式化说明 Causal Language Modeling 的预训练目标。重点是 sequence probability 的 autoregressive factorization、next-token prediction 与 token-level negative log-likelihood / cross-entropy 之间的关系，并区分 vocabulary size、sequence length、logits、probability distribution、target token 以及不同粒度的 loss。最后说明 teacher forcing 与 causal masking 如何使训练阶段能够并行计算整段序列的 loss，而 generation 仍必须逐 token 进行。

1 Introduction

前四章给出了 token sequence 如何进入 decoder-only Transformer、如何经过 causal Self-Attention 与 Feed-Forward Network 更新 residual stream，以及如何由最终 hidden states 产生 vocabulary logits。Pretraining 赋予这条 computation graph 一个明确的统计任务：从文本语料中学习 token sequence 的 probability distribution。对一个 tokenized sequence $x = (x_1, \dots, x_T)$ ，模型并不试图一次性从一个无结构的空间中挑选整段文本；它学习在每个位置上，根据已经出现的 prefix 为下一个 token 分配概率。

这一目标是 modern autoregressive language model 的核心。它把无标注文本转化为监督信号：同一段文本同时提供 context 和 target。Neural probabilistic language modeling 的目标正是对 token sequence 的 joint probability 建模 [1]；GPT-style models 将这一原则扩展为大规模 decoder-only Transformer 的 pretraining objective [2]。

本章沿用 Chapter 1 的 token notation。令 $\mathcal{V}$ 为 vocabulary，令 $V = |\mathcal{V}|$ 为其大小； T 为某个 sequence 的 token length，而不是 vocabulary 的属性。对于 batch，令 B 为 sequence 数。Chapter 2 中的 language-model head 对每个 batch item 与 position 产生一个长度为 V 的 logit vector；本章说明这些 vectors 如何变成条件概率和训练 loss。

2 Autoregressive Language Modeling

Autoregressive Language Modeling 假定序列从左到右展开。在第 t 个预测位置，模型只允许使用严格位于其左侧的 token prefix，记为 $x_{\{<t\}} = (x_1, \dots, x_{t-1})$ 。模型参数为 θ 时，对 vocabulary 内任意 token v ，条件分布写为

$$p_{\theta}(v \mid x_{\{<t\}}), \quad v \in \mathcal{V}. \quad (1)$$

这不是对一个已知 target 的单一分数，而是 vocabulary 上的完整 distribution。它满足非负性与归一化条件： $p_{\theta}(v \mid x_{\{<t\}}) \geq 0$ ，且 $\sum_{v \in \mathcal{V}} p_{\theta}(v \mid x_{\{<t\}}) = 1$ 。在 text generation 中，该 distribution 表示下一 token 的候选；在 pretraining 中，语料给出实际发生的 target token，模型因此能评估其 assigned probability。

对于可变长度文本，sequence termination 本身也必须成为概率模型的一部分。实际实现通常使用一个 end-of-sequence token，令模型在适当位置预测它。为避免在本章的主要推导中增加边界符号，以下将 $x_1, \dots, x_T$ 看成已经包含所需 boundary convention 的 token sequence。

3 Sequence Probability Factorization

任何 joint distribution 都可以通过 probability chain rule 分解。Autoregressive model 以 parameterized conditionals 近似这一定律：

$$p_{\theta}(x_1, \dots, x_T) = \prod_{t=1}^T p_{\theta}(x_t | x_{\{<t\}}). \quad (2)$$

这个 factorization 的含义不是 token 彼此独立；恰恰相反，第 t 项可以依赖完整 prefix。它规定的是依赖方向：模型可以把已出现的 token 压缩进 hidden state，却不能在预测 x_t 时读取 x_t 或任何 future token。Chapter 3 的 causal Attention Mask 是 Transformer 实现这一信息约束的机制。乘积形式适合表达 probability，却不适合数值优化。取 logarithm 后，sequence log-probability 变为各位置条件 log-probability 的和：

$$\log p_{\theta}(x_1, \dots, x_T) = \sum_{t=1}^T \log p_{\theta}(x_t | x_{\{<t\}}). \quad (3)$$

因此，最大化 whole-sequence likelihood 与最大化每个 observed next token 的 conditional log-probability 是同一个目标。该等价关系是 next-token prediction 从局部 training signal 导出 sequence-level probability model 的原因。

4 Next-Token Prediction

在训练代码中，next-token prediction 通常通过一个 shift 实现。若引入 beginning-of-sequence token x_0 ，则模型输入为 $(x_0, x_1, \dots, x_{\{T-1\}})$ ，第 $t-1$ 个 position 的 output 用来预测 target x_t 。没有显式 x_0 的实现也常将一个 packed sequence 的 inputs 与 labels 右移一格：position t 的 logits 与 position $t+1$ 的 token 对齐。两种写法的共同语义都是：target x_t 的 prediction 只以 $x_{\{<t\}}$ 为条件。

这里的 target token 是一个 integer ID，不是 one-hot vector，也不是模型生成出的 token。对于 batch item b 和 supervised position t ，记其 target 为 $y_{b,t} \in \{0, \dots, V-1\}$ 。在最常见的 right-shift convention 中， $y_{b,t}$ 就是 input sequence 中的下一个 token。实现中的 off-by-one error 会改变训练目标本身，而不是造成一个可由 optimizer 自动修正的小误差。

5 Logits and Token Probabilities

Chapter 2 将 language-model head 的输出记为 $O \in R^{B \times T \times V}$ 。其中 $O_{b,t,:}$ 是长度为 V 的 vector，称为 logits。它们是 unnormalized scores，而不是 probabilities。对固定 (b, t) ，softmax 定义

$$p_{\theta}(v | x_{b,<t}) = \frac{\exp(O_{b,t,v})}{\sum_{j=0}^{V-1} \exp(O_{b,t,j})}. \quad (4)$$

softmax 将任意实值 vector 映射到 vocabulary simplex。给所有 logits 加同一个常数不会改变 Equation 4，因此 logit 的绝对零点没有概率意义；不同 vocabulary item 之间的 relative score 才决定分布。 V 决定 vector 的宽度， T 决定需要预测的位置数，两者不可互换。

对象	符号或 shape	含义
Vocabulary size	$V = \mathcal{V} $	可取 token ID 的数量，也是每个 logit vector 的宽度。
Sequence length	T	一个 sequence 的 token position 数；它随 tokenized text 改变。
Logits	$O_{b,t,:} \in R^V$	一个 prediction position 的 unnormalized scores。
Probability distribution	$p_{\theta}(\cdot x_{b,<t})$	经 softmax 归一化后定义在 vocabulary 上的 categorical distribution。
Target token	$y_{b,t}$	用于选取一个 probability 的 observed integer token ID。
Token-level loss	$\ell_{b,t}$	模型赋予一个 observed target 的 negative log-probability。
Sequence / batch loss	$\mathcal{L}_b, \mathcal{L}_{\text{batch}}$	按明确 reduction rule 聚合 token losses 得到的 scalar。

Table 1: Language-model training 中容易混淆的对象。target 是一个 index；logits 与 probabilities 是长度为 V 的 vectors；loss 将这些对象归约为 scalars。

6 Cross-Entropy and Negative Log-Likelihood

对一个 supervised position，定义 empirical target distribution $q_{b,t}$ ，使它把全部 probability mass 放在 $y_{b,t}$ 上。从这个 one-hot target distribution 到 model distribution 的 cross-entropy 为

$$\ell_{b,t} = - \sum_{v=0}^{V-1} q_{b,t}(v) \log p_{\theta}(v | x_{b,<t}) = - \log p_{\theta}(y_{b,t} | x_{b,<t}). \quad (5)$$

最后一个表达式就是 token-level negative log-likelihood (NLL)。当 target 为 one-hot 时，cross-entropy 与 NLL 指向同一个数值 training term。二者的概念侧重点仍有区别：cross-entropy 强调两个 distributions 的比较，NLL 强调模型赋给 observed sample 的 probability。标准 information theory 将 cross-entropy 与 optimal code length 和 likelihood 联系起来 [3]。

将 softmax 展开后，可得到直接以 logits 表示的形式：

$$\ell_{b,t} = -O_{b,t,y_{b,t}} + \log \sum_{v=0}^{V-1} \exp(O_{b,t,v}). \quad (6)$$

第一项鼓励 correct token 具有较高 logit；第二项通常称为 log-sum-exp，它纳入所有 competing vocabulary items。对任一 logit 的 derivative 为

$$\frac{\partial \ell_{b,t}}{\partial O_{b,t,v}} = p_{\theta}(v | x_{b,<t}) - \delta_{v,y_{b,t}}, \quad (7)$$

其中当 $v = y$ 时 $\delta_{v,y}$ 为一，否则为零。因此，gradient descent 会提高 target logit，并按 predicted probability 的大小降低 competing logits。这个局部表达式也说明，若要计算 exact full-softmax loss，language-model head 必须先产生全部 V 个 scores。

在数值实现中，不应先直接计算 $\exp(O_{b,t,v})$ 。令 $a = \max_v O_{b,t,v}$ ，则有代数上等价的 identity

$$\log \sum_v \exp(O_{b,t,v}) = a + \log \sum_v \exp(O_{b,t,v} - a) \quad (8)$$

它使 exponent 的 arguments 不大于零。实际 framework 通常把这一 stable log-sum-exp calculation 与 target gather 融合，而不是为 loss 单独 materialize 一个 probability tensor。

7 Teacher Forcing and Causal Masking

Teacher forcing 指训练时提供的 prefix 来自 observed corpus, 而不是模型先前 samples 的序列。该术语早于 Transformer [4], 但其原则可以直接沿用: 对 x_t 评分时, 模型从 data 接收 $x_{\{<t\}}$, 并对已知 target 计算 token-level loss。因此, sequence 中每个 eligible token position 都能提供 learning signal。

Causal masking 使 teacher forcing 对 autoregressive modeling 保持统计上的有效性。在 decoder-only attention layer 中, position t 只能 attend 到允许的 earlier positions; future tokens 在 attention softmax 前被 masked。该 mask 阻止 representation 编码自己将被要求预测的答案。它本身并不定义 loss: loss 由 target alignment 和 token-level cross-entropy 定义。原始 Transformer 在 decoder self-attention 中使用 mask, 使 prediction 不依赖 subsequent positions [5]。

这种 causal dependency 并不要求 sequential training loop。给定完整 input tensor, 所有 $B \times T$ positions 的 projections、masked attention scores、FFNs、language-model head 和 token losses 都能用 batched tensor operations 计算。mask 为这一 parallel computation 的每一行强制相同的 allowed-prefix 规则。generation 则不同, 因为 next input token 尚未知: 模型先对当前 prefix 产生 distribution, 必须选取并 append 一个 token, 才能计算下一步的 distribution。因此, parallel training 与 sequential generation 是同一 conditional model 的两种 execution regimes, 而不是 autoregression 的两种相互竞争的定义。

8 Loss Averaging Across Tokens and Batches

一个完整 sequence 的 NLL 是各 token-level losses 的和:

$$\mathcal{L}_b = -\log p_{\theta}(x_{b,1}, \dots, x_{b,T}) = \sum_{t=1}^T \ell_{b,t}. \quad (9)$$

训练通常报告并对 mean loss 求导, 而不是使用 raw sum。令 $m_{b,t} \in \{0, 1\}$ 表示 position (b, t) 是否具有 valid supervised target, 并令 $N = \sum_{b=1}^B \sum_{t=1}^T m_{b,t}$ 。则 token-averaged batch loss 为

$$\mathcal{L}_{\text{batch}} = \frac{1}{N} \sum_{b=1}^B \sum_{t=1}^T m_{b,t} \ell_{b,t}. \quad (10)$$

loss mask m 与 attention mask 不同。causal attention mask 约束 information flow; loss mask 决定哪些 output positions 贡献给 scalar objective。padding positions、没有 shifted target 的 positions, 以及被有意排除的 labels 都应取 $m_{b,t} = 0$ 。除以 N 使每个 valid token 即使处于 effective length 不同的 sequence 中也有相同权重。相反, 先对每个 sequence 取 mean 再对 sequences 取 mean, 会让短 sequence 与长 sequence 权重相同。两种 reduction 并非有一个必然错误, 但选择会改变 objective, 必须与 reported loss 一同记录。

9 Perplexity

使用 natural logarithm 与 token-average NLL $\mathcal{L}_{\text{batch}}$ 时, perplexity 定义为

$$\text{PPL} = \exp(\mathcal{L}_{\text{batch}}). \quad (11)$$

对没有 masked positions 的单个 sequence, 它是模型赋给 observed tokens 的 probabilities 的 reciprocal geometric mean。数值接近一表示模型对 evaluated token stream 赋予了较高 probability; 更大的数值表示更高的 average uncertainty。若以 base-two logarithm 用 bits 计算 cross-entropy, 等价表达式为 2^H 。

Perplexity 的价值在于把 average log-loss 转换为可解释的 effective branching factor, 但它不是 tokenizer-independent 的 language-understanding measure。Chapter 1 已指出 tokenization 同时改变 event space 与 prediction positions 的数量。若使用不同 vocabulary、normalization policy 或 loss mask 评估, 得到的 token-level PPL 不能直接比较。更低的 PPL 也不会单独规定 generation 时应使用何种 decoding rule。

10 Training Objective versus Generation

在 pretraining 中, likelihood evaluation 提出的是一个不涉及 counterfactual 的问题: 给定来自 corpus 的 true prefix, 模型给 true next token 分配了多大 probability? 完整 sequence 虽在 memory 中可用, 但 causal masking 确保每个 prediction 只使用其 permitted prefix。最终输出是对大量这类 conditional probabilities 聚合得到的 scalar objective。

在 generation 中, 只有 prefix 可用。模型产生 $p_{\theta}(\cdot | x_{\{<t\}})$, 再由 decoding rule 选择或 sample 一个 token 来扩展 prefix。新产生的 token 随后成为下一步 context 的一部分。Maximum likelihood training 规定的是如何拟合 conditional distribution; 它并不规定 greedy selection、temperature scaling 或任何其他 decoding policy。这些 inference choices 属于后续章节, 但此处的区分至关重要: token-level training loss 在 data prefixes 下评估 probabilities, 而 generation 在 model-produced prefixes 下构造 trajectory。

11 Implementation Contracts

一个最小的 Causal Language Modeling training step 应显式给出四个对齐的 tensors: shape 为 $B \times T$ 的 input token IDs、shape 为 $B \times T$ 的 target IDs、shape 为 $B \times T \times V$ 的 logits, 以及 shape 为 $B \times T$ 的 loss mask。一个 supervised position 的 target 必须是对应 logit 所表示的 input prefix 紧随其后的 token。凡 loss mask 为一处, target IDs 必须是 $[0, V - 1]$ 内的 integer indices, 而不是 floating-point probabilities。

loss implementation 应消费 raw logits, 使用 stable fused cross-entropy kernel, 并且只在 valid targets 上执行 reduction。测试应在短小的手写 sequence 上显式检查 shift, 验证 masked targets 的贡献为零, 并将小 batch 与 high precision 下的直接 probability calculation 对照。causality test 应修改 future input token, 并确认 earlier positions 的 logits 不变。这些 contracts 将数学 objective 转化为可复现的 tensor program。

12 Summary

Causal Language Modeling 通过将 joint probability 分解为 next-token conditionals, 为 token sequence 赋予 probability。decoder-only Transformer 在每个 position 产生一个 vocabulary-sized logit vector; softmax 将它变为 conditional distribution, 而 observed target 的 negative log-probability 给出 token-level cross-entropy。对这些 terms 求和得到 sequence NLL, 按 loss mask 做 token average 则得到常用 batch loss 与对应的 perplexity。

Teacher forcing 在训练中提供 observed prefixes, causal masking 则阻止这些 prefixes 包含 future answers。这使整个 batch 的 position-wise losses 可以并行计算。Autoregressive generation 仍是 sequential 的, 因为每个新选定 token 都会改变下一步 prefix。该 objective 的形式很简单, 但 tensor alignment、masking、reduction convention 和 numerical implementation 共同定义实际的 pretraining problem。

References

- [1] Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin, “A Neural Probabilistic Language Model,” *Journal of Machine Learning Research*, vol. 3, pp. 1137–1155, 2003, [Online]. Available: <https://jmlr.org/papers/v3/bengio03a.html>
- [2] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language Models are Unsupervised Multitask Learners,” technical report, 2019. [Online]. Available: <https://cdn.openai.com/better-language-models/language-models.pdf>
- [3] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley-Interscience, 2006. doi: 10.1002/047174882X.
- [4] R. J. Williams and D. Zipser, “A Learning Algorithm for Continually Running Fully Recurrent Neural Networks,” *Neural Computation*, vol. 1, no. 2, pp. 270–280, 1989, doi: 10.1162/neco.1989.1.2.270.
- [5] A. Vaswani *et al.*, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems 30*, 2017, pp. 5998–6008.